

C++ 프로그래밍

(예외 처리와 형 변환)

10주차

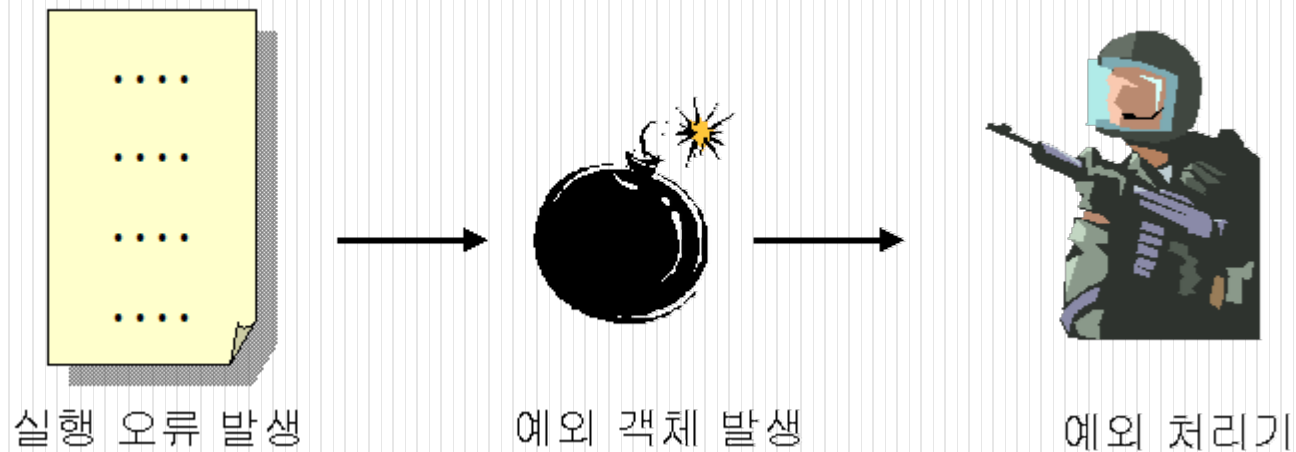
목차

- 예외 처리의 개념
- 예외 처리기 구조
- 예외 전달
- 다중 catch문장
- 자신의 예외 클래스 작성
- 형 변환

예외 처리의 개념

- 예외란?
 - 잘못된 코드, 부정확한 데이터, 예외적인 상황에 의하여 발생하는 오류

예) 0으로 나누는 것과 같은 잘못된 연산이나 배열의 인덱스가 한계를 넘을 수도 있고, 디스크에서는 하드웨어 에러가 발생할 수 있다.



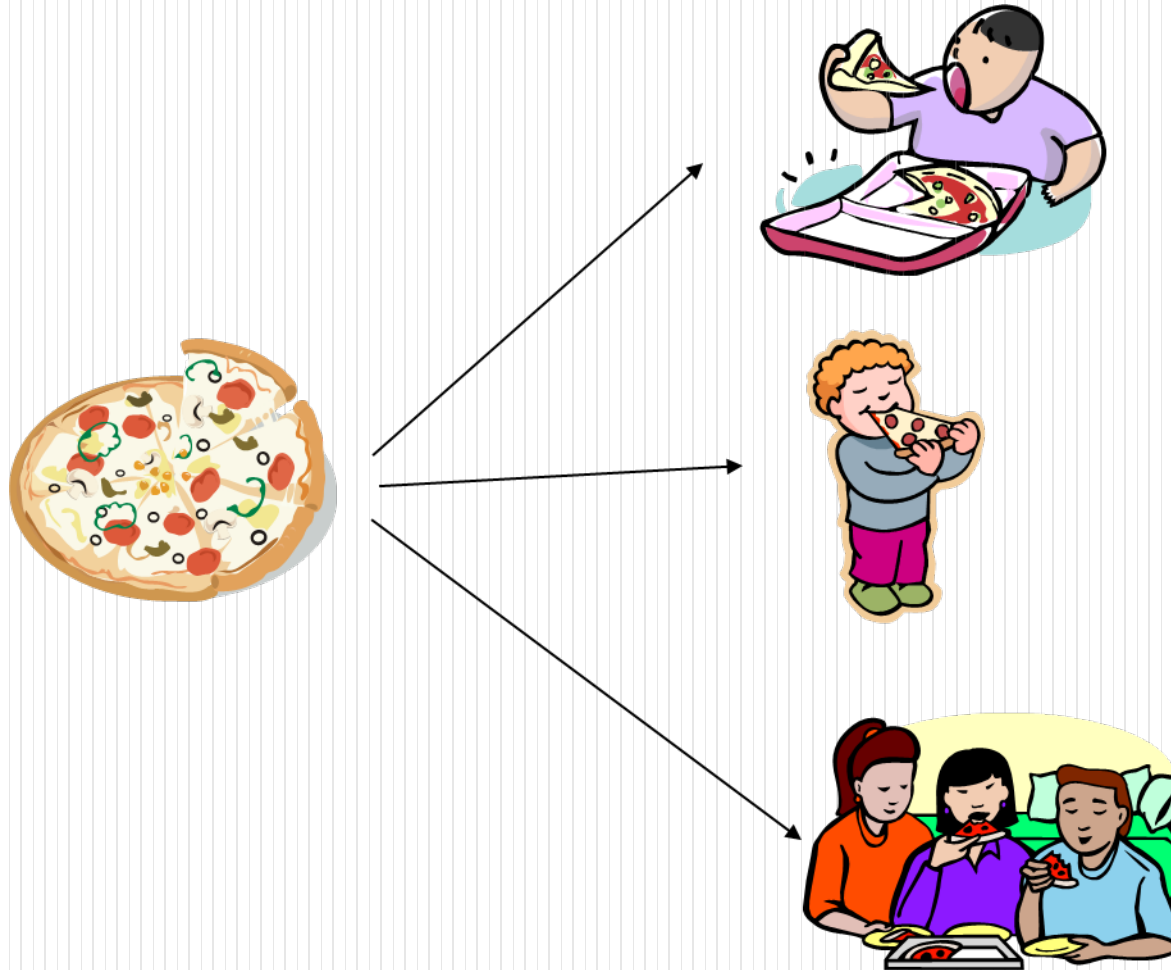
예외 처리의 개념 (계속)

- 예외는 처리하는 것이 좋다.



예외 처리의 개념 (계속)

- 피자 나누기 프로그램



예외 처리의 개념 (계속)

- 피자 나누기 프로그램 (계속)

```
#include <iostream>
using namespace std;
int main()
{
    int pizza_slices = 0;
    int persons = -1;
    int slices_per_person=0;

    cout << "피자 조각수를 입력하시오: ";
    cin >> pizza_slices;
    cout << "사람수를 입력하시오: ";
    cin >> persons;
    slices_per_person = pizza_slices / persons;
    cout << "한사람당 피자는" << slices_per_person << "입니다." << endl;

    return 0;
}
```



피자 조각수를 입력하시오: 12
사람수를 입력하시오: 4
한사람당 피자는 3입니다.
계속하려면 아무 키나 누르십시오 ...

예외 처리기 구조

- 예외 처리기

```
try {
```



throw 예외;

```
}
```

```
catch {
```



```
}
```

try블록은 예외가 발생할 수 있는 블록입니다.

throw는 예외를 던진다.

catch 블록은 try 블록에서 예외가 발생하면 처리합니다.

예외 처리기 구조 (계속)

- 예제

```
int main()
{
    int pizza_slices = 0;
    int persons = -1;
    int slices_per_person=0;
    try
    {
        cout << "피자조각 수를 입력하시오: ";
        cin >> pizza_slices;
        cout << "사람수를 입력하시오: ";
        cin >> persons;

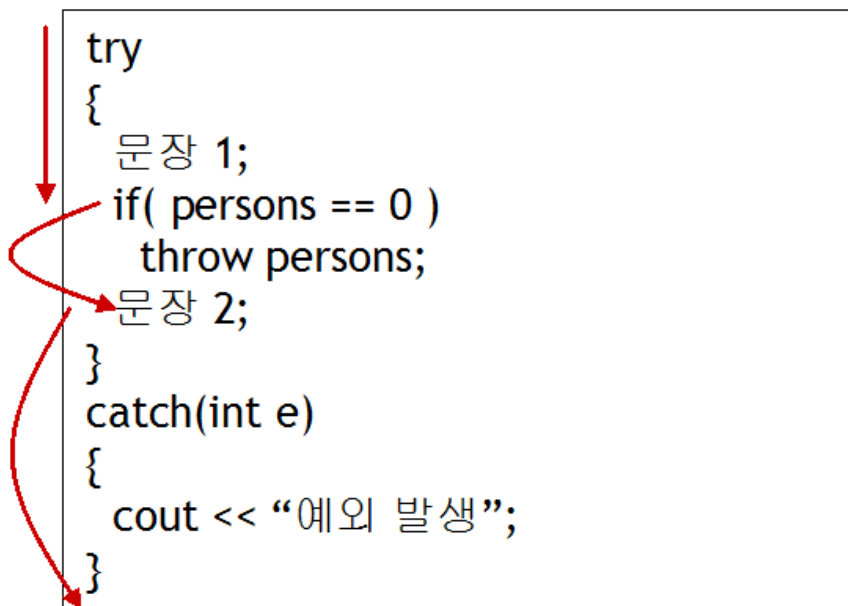
        if (persons == 0)
            throw persons;
        slices_per_person = pizza_slices / persons;
        cout << "한사람당 피자는" << slices_per_person << "입니다." << endl;
    }
    catch(int e)
    {
        cout << "사람이" << e << " 명입니다." << endl;
    }
    return 0;
}
```

피자 조각수를 입력하시오: 12
사람수를 입력하시오: 4
한사람당 피자는 3입니다.
계속하려면 아무 키나 누르십시오 ...

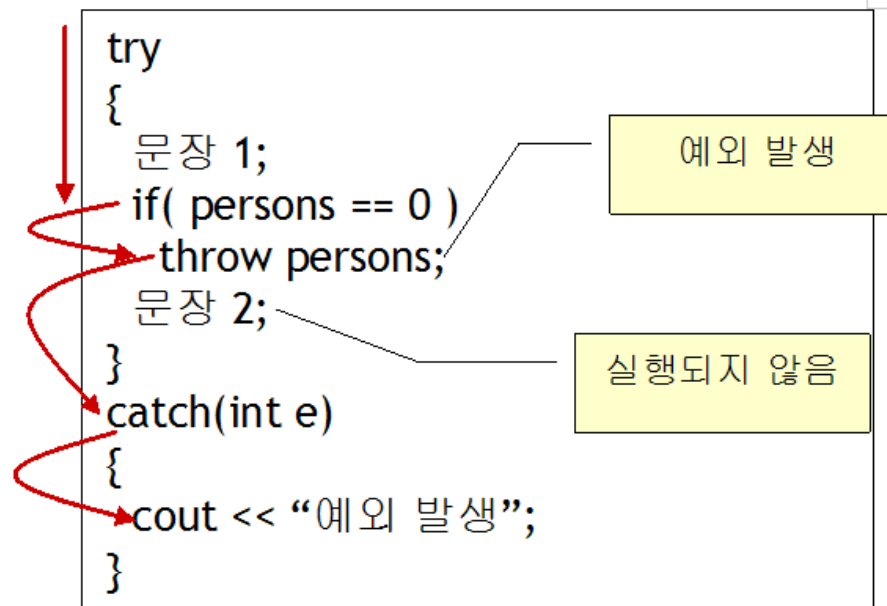
피자 조각수를 입력하시오: 12
사람수를 입력하시오: 0
사람이 0 명 입니다.
계속하려면 아무 키나 누르십시오 ...

예외 처리기 구조 (계속)

- try/catch 블록에서의 실행 흐름



예외가 발생하지 않은 경우

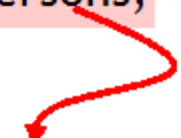


예외가 발생한 경우

예외 처리기 구조 (계속)

- catch블록의 매개 변수

```
try
{
    문장 1;
    if( persons == 0 )
        throw persons;
    문장 2;
}
catch(int e)
{
    cout << “예외 발생”;
}
```



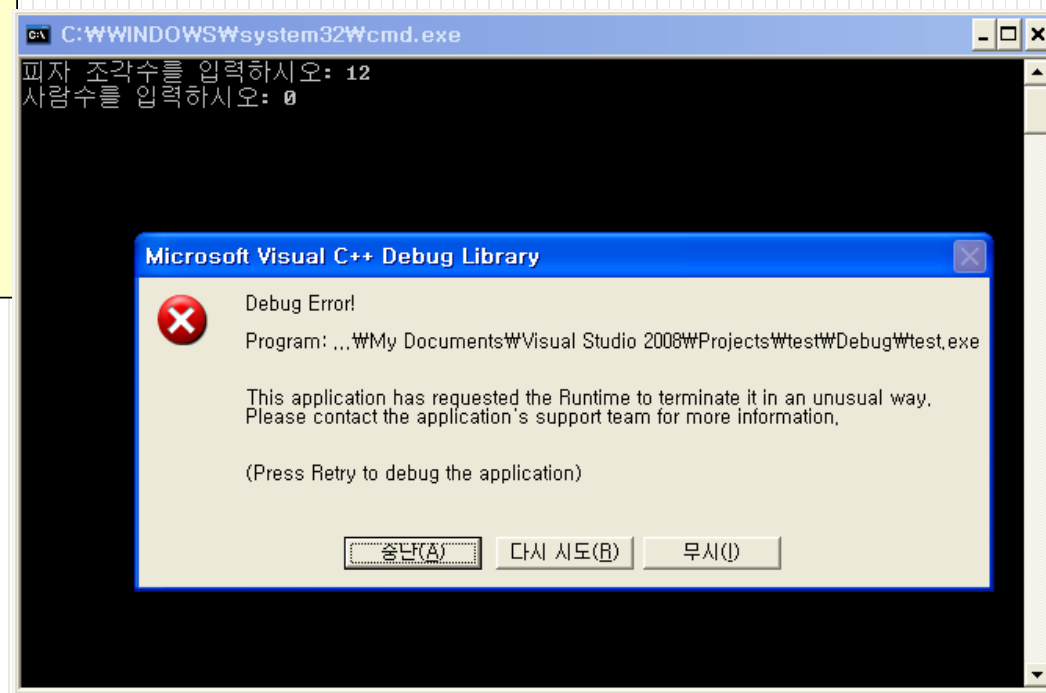
예외 처리기의 매개 변수

예외 처리기 구조 (계속)

- 타입이 일치되는 예외만 처리

```
try
{
    int person =0;
    ...
    if (persons == 0)
        throw persons;
    ...
}
catch(char e)
{
    cout << "사람이 " << e << " 명 입니다. "<< endl;
}
```

타입이 일치하지 않음



예외 처리기 구조 (계속)

- 모든 타입의 예외를 잡고 싶으면

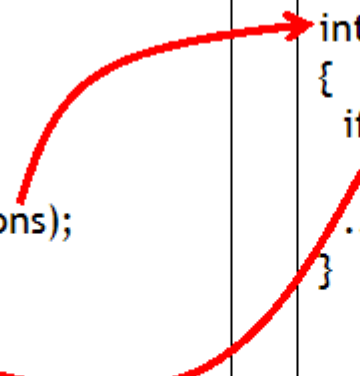
```
catch(...)  
{  
    // 모든 예외를 잡아서 처리할 수 있다.  
}
```

예외 전달

- 예외 전달

```
int main()
{
    try
    {
        dividePizza(slices, persons);
    }
    catch(int e)
    {
        cout << "예외 발생";
    }
}
```

```
int dividePizza(int slices, int persons)
{
    if( persons == 0 )
        throw persons;
    ...
}
```



예외 전달 (계속)

● 예제

```
#include <iostream>
using namespace std;
int dividePizza(int pizza_slices, int persons);

int main()
{
    int pizza_slices = 0;
    int persons = 0;
    int slices_per_person=0;

    try
    {
        cout << "피자 조각수를 입력하시오: ";
        cin >> pizza_slices;
        cout << "사람수를 입력하시오: ";
        cin >> persons;
        slices_per_person =dividePizza(pizza_slices, persons);
        cout << "한사람당 피자는 " << slices_per_person << "입니다." << endl;
    }
```

```
catch(int e)
{
    cout << "사람이 " << e << "명 입니다." << endl;
}
return 0;
}
int dividePizza(int pizza_slices, int persons)
{
    if (persons == 0)
        throw persons;
    return pizza_slices / persons;
}
```

정상적인 실행결과

피자 조각수를 입력하시오: 12
사람수를 입력하시오: 4
한사람당 피자는 3입니다.
계속하려면 아무 키나 누르십시오 . . .

예외 발생 실행결과

피자 조각수를 입력하시오: 12
사람수를 입력하시오: 0
사람이 0명 입니다.
계속하려면 아무 키나 누르십시오 . . .

예외 전달 (계속)

- 예외를 처리하고 다시 보내고자 할때

```
int dividePizza(int pizza_slices, int persons)
{
    try
    {
        if (persons == 0)
            throw persons;
        return pizza_slices / persons;
    }
    catch(int e)
    {
        cout << "사람이 " << e << " 명 입니다(dividePizza). "<< endl;
        throw;
    }
}
```

예외 발생 실행결과

피자 조각수를 입력하시오: 12

사람수를 입력하시오: 0

사람이 0 명 입니다(dividePizza).

사람이 0 명 입니다.

계속하려면 아무 키나 누르십시오 ...

예외 전달 (계속)

- 함수 헤더에 예외 명시

```
int dividePizza(int s, int p) throw () // 예외를 던지지 않는다.  
int dividePizza(int s, int p) throw (..) // 예외를 던진다. 타입은 지정하지 않음  
int dividePizza(int s, int p) throw (int) // int 타입의 예외를 던진다.  
int dividePizza(int s, int p) throw (int, double)
```


다중 catch문장

- 다중 catch문장
 - 하나의 try블록에서는 여러 개의 throw문장을 가질 수 있다.
 - 여러 가지 타입의 값을 처리하려면 여러 개의 catch블록을 두어야 한다.
 - 예를 들어서 피자 나누기 예제에서 사람 수가 0이 될 수도 있고 사람수가 음수가 될 수도 있다. 이것을 구분하여서 처리하려면 다음과 같이 두개의 catch블록을 정의하여야 한다.

다중 catch문장 (계속)

● 예제

```
#include <iostream>
using namespace std;

int main()
{
    int pizza_slices = 12;
    int persons = 0;
    int slices_per_person=0;

    try {
        cout << "피자 조각수를 입력하시오: ";
        cin >> pizza_slices;
        cout << "사람수를 입력하시오: ";
        cin >> persons;

        if( persons < 0 ) throw "negative"; // 예외 발생!
        if( persons == 0 ) throw persons; // 예외 발생!
        slices_per_person = pizza_slices / persons;
        cout << "한사람당 피자는 " << slices_per_person << "입니다." << endl;
    }
    catch (const char *e) { // char 타입의 예외만 처리
        cout << "오류: 사람수가 " << e << "입니다" << endl;
    }
    catch (int e) { // int 타입의 예외만 처리
        cout << "오류: 사람이 " << e << "명입니다." << endl;
    }
    return 0;
}
```

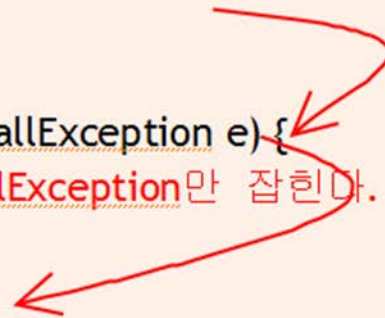
예외 발생 실행결과

피자 조각수를 입력하시오: 12
사람수를 입력하시오: 0
사람이 0 명 입니다.
계속하려면 아무 키나 누르십시오 . . .

다중 catch문장 (계속)

- 구체적인 예외를 먼저 잡는다.

```
try {  
    getInput();  
}  
catch(TooSmallException e) {  
    //TooSmallException만 잡힌다.  
}  
catch(...) {  
    //TooSmallException을 제외한 나머지 예외들이 잡힌다.  
}
```



다중 catch문장 (계속)

- 구체적인 예외를 나중에 잡는다.

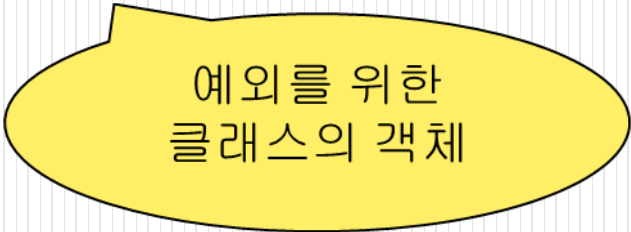
```
try {  
    getInput();  
}  
catch(...) {  
    //모든 예외들이 잡힌다.  
}  
catch(TooSmallException e) {  
    //아무 것도 잡히지 않는다!  
}
```



자신의 예외 클래스 작성

- 예외 클래스 작성
 - Throw문장은 클래스 타입의 객체도 던질 수 있다.
 - 예외 클래스도 단지 하나의 클래스에 불과

예) throw **NoPersonException**(persion);



예외를 위한
클래스의 객체

자신의 예외 클래스 작성 (계속)

- 예제

```
#include <iostream>
using namespace std;
```

```
class NoPersonException
{
public:
    NoPersonException();
    NoPersonException(int p) { persons = p; };
    int get_persons() { return persons; };
private:
    int persons;
};
```

```
int main()
{
    int pizza_slices = 12;
    int persons = -1;
    int slices_per_person=0;

    try {
        cout << "피자 조각수를 입력하시오: ";
        cin >> pizza_slices;
        cout << "사람수를 입력하시오: ";
        cin >> persons;
        if( persons <= 0 ) throw NoPersonException(persons);        // 예외 발생!
        slices_per_person = pizza_slices / persons;
        cout << "한사람당 피자는 " << slices_per_person << "입니다." << endl;
    }
    catch (NoPersonException e)
    {
        cout << "오류: 사람이 " << e.get_persons() << "명 입니다" << endl;
    }
    return 0;
}
```

예외 발생 실행결과

피자 조각수를 입력하시오: 12

사람수를 입력하시오: 0

사람이 0 명 입니다.

계속하려면 아무 키나 누르십시오 . . .

자신의 예외 클래스 작성 (계속)

- 상속 관계에 있는 예외 클래스

```
#include <iostream>
using namespace std;

class ParentException
{
public:
    void display() { cout << "ParentException" << endl; }
};

class ChildException : public ParentException
{
public:
    void display() { cout << "ChildException" << endl; }
};
```

```
int main()
{
    try {
        throw ChildException();
    }
    catch (ParentException& e)
    {
        e.display();
    }
    catch (ChildException& e)
    {
        e.display();
    }
    return 0;
}
```

예외 발생 실행결과

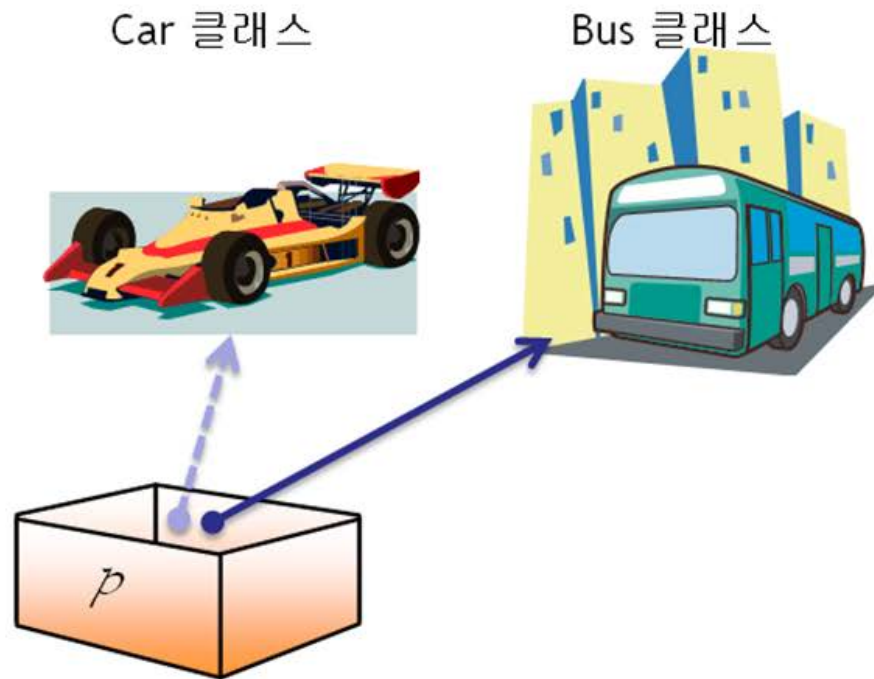
ParentException

계속하려면 아무 키나 누르십시오 . . .

형 변환

- 형 변환

```
double f = 3.141592;  
int i = (int) f;
```



형변환은 변수의
타입 (type)을 변
경하는 연산입니
다.



형 변환 (계속)

- C++의 형 변환 연산자

형변환 연산자	의미
<code>static_cast</code>	기본 타입의 변환이나 상속 관계에 있는 클래스 포인터를 변환할 때 사용
<code>dynamic_cast</code>	상속 관계에 있는 클래스 포인터를 안전하게 변환할 때 사용, 실행 시간에 식별 가능
<code>const_cast</code>	상수 속성을 변경
<code>reinterpret_cast</code>	관련없는 포인터 사이의 무조건 변환, 정수형과 포인터 사이의 변환

형 변환 (계속)

- static_cast
 - 컴파일 시간에 논리적으로 타당한 변환만을 수행한다.

형식

static_cast<타입> (대상)

예를 들어서 double형의 변수 f를 int형으로 변환하려면 다음과 같은 문장을 작성하면 된다.

```
i = static_cast<int>(f);
```

형 변환 (계속)

- 예제

```
#include <iostream>
using namespace std;

int main()
{
    int i = 9;
    double f = 3.141592;
    int *pi ;
    double *pf = &f;

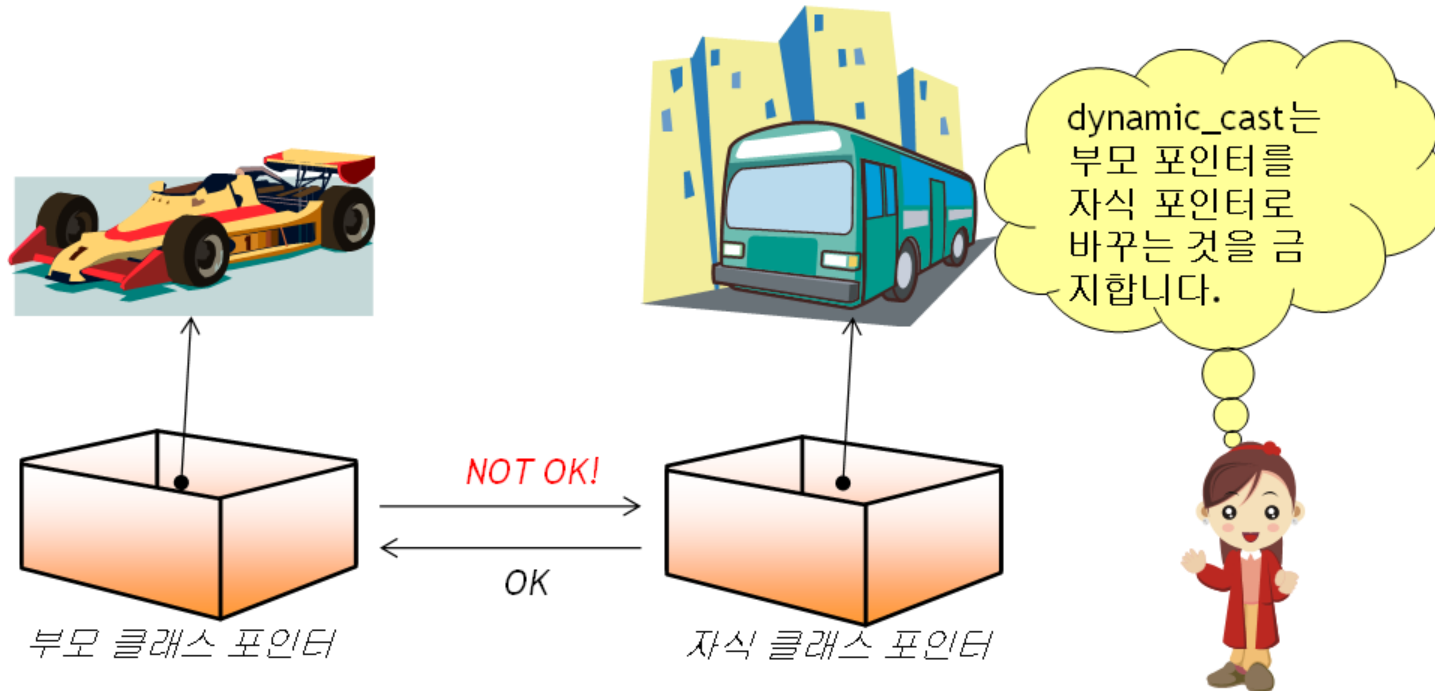
    i = static_cast<int>(f);          // OK
    pi = static_cast<int*>(pf);        // 오류!
    return 0;
}
```

실행 결과

```
l>c:\users\chun\documents\visual studio 2008\projects\test\test\test.cpp(12) : error C2440:
'static_cast' : 'double *'에서 'int *'(으로 변환할 수 없습니다
l>          가리킨 형식이 관련이 없습니다. 변환하려면 reinterpret_cast, C 스타일캐스트 또는 함
수스타일캐스트가 필요합니다
```

형 변환 (계속)

- dynamic_cast
 - 기본적으로 상향 형변환(upcast)은 허용하지만 하향 형 변환(downcast)은 허용하지 않는다.



형 변환 (계속)

- `dynamic_cast` (계속)
 - 실행 시간에 포인터가 가리키는 객체의 타입을 보고 판단하여서 변환이 올바르게 반환된 타입이 반환된다. 그렇지 않으면 `NULL`이 반환된다.
 - 이 기능을 사용하려면 부모 클래스가 가상 함수를 사용하고 있어야 한다.

형 변환 (계속)

- 예제

```
class Car
{
};
class Bus : public Car
{
};

int main()
{
    Bus *pBus1 = new Bus;
    Car *pCar1 = dynamic_cast<Car *>(pBus1);        // OK!

    Car *pCar2 = new Car;
    Bus *pBus2 = dynamic_cast<Bus *>(pCar2);        // ① 컴파일 오류!

    Car *pCar3 = new Bus;
    Bus *pBus3 = dynamic_cast<Bus *>(pCar3);        // ② 컴파일 오류!

    return 0;
}
```

형 변환 (계속)

- `const_cast`
 - `const_cast`는 타입에서 `const`속성을 제거하거나 추가한다.

```
#include <iostream>
using namespace std;
```

```
void display(char *s)
{
    cout << s << endl;
}
```

```
int main()
{
    const char *saying = "A bad workman (always) blames his tools";
    display(const_cast<char *>(saying));
    return 0;
}
```

`saying`의 타입에서 `const`를 제거한다.

형 변환 (계속)

- reinterpret_cast
 - 어떤 포인터 타입이라도 다른 포인터 타입으로 변환

```
class Car
```

```
{
```

```
};
```

```
class Box
```

```
{
```

```
};
```

```
int main()
```

```
{
```

```
char *pc;
```

```
pc = reinterpret_cast<char *>(0x10000ef);
```

정수를 char*로 변환

```
Car *pCar1 = new Car;
```

```
Box *pBox1 = reinterpret_cast<Box *>(pCar1);
```

Car 클래스 포인터를

Box 클래스 포인터로 변

환

```
return 0;
```

```
}
```


타입 정보

- 타입 정보
 - typeid연산자는 실행 시간에 객체의 타입을 식별할 수 있다.
 - typeid가 반환하는 값은 type_info에 대한 참조자 이다.

타입 정보

- 예제

```
class Car {  
public:  
    virtual void display() {}  
};  
  
class SportsCar : public Car {  
};  
  
int main() {  
    SportsCar* pd = new SportsCar;  
    Car* pb = pd;  
    cout << typeid( pb ).name() << endl;  
    cout << typeid( *pb ).name() << endl;  
    cout << typeid( pd ).name() << endl;  
    cout << typeid( *pd ).name() << endl;  
    delete pd;  
    return 0;  
}
```

타입 정보 출력

실행 결과

```
class Car *  
class SportsCar  
class SportsCar *  
class SportsCar
```